

MFarm Client — Documentación Técnica y Manual de Despliegue

Versión del documento: 1.0 — Junio 2026
Versión del proyecto: 1.26.1
Plataforma: React SPA (Single Page Application)

Tabla de Contenidos

- 1. Descripción General
- 2. Stack Tecnológico
- 3. Requisitos del Entorno
- 4. Configuración del Entorno de Desarrollo
- 5. Variables de Entorno
- 6. Estructura del Proyecto
- 7. Arquitectura de la Aplicación
- 8. Módulos y Rutas
- 9. Gestión de Estado (Redux)
- 10. Autenticación y Autorización
- 11. Cliente HTTP (Axios)
- 12. Internacionalización (i18n)
- 13. Convenciones de Código
- 14. Flujo de Trabajo Git
- 15. Manual de Despliegue en Producción

1. Descripción General

MFarm Client es el frontend de un sistema de gestión porcícola (pig farming management system). Permite a granjas gestionar cerdos, grupos, alimentación, gestación, partos, laboratorio, almacén, ventas, reportes financieros y configuraciones.

La aplicación es una **SPA** construida con React 18 y TypeScript, que se comunica con un backend REST + WebSocket. Soporta múltiples granjas, roles de usuario diferenciados y tres idiomas (español, inglés, portugués).

2. Stack Tecnológico

Framework y lenguaje

Tecnología	Versión	Propósito
React	18.3.1	UI framework principal
TypeScript	5.6.3	Tipado estático

Tecnología	Versión	Propósito
Create React App (CRA)	5.0.1 (react-scripts)	Toolchain de build (Webpack interno)

Estado y datos

Tecnología	Versión	Propósito
Redux Toolkit	2.2.8	Gestión de estado global
React-Redux	9.1.2	Integración React ↔ Redux
Reselect	5.1.1	Selectores memoizados
Axios	1.7.7	Cliente HTTP
Socket.IO Client	4.8.3	Notificaciones en tiempo real (WebSocket)

Formularios y validación

Tecnología	Versión	Propósito
Formik	2.4.6	Gestión de formularios
Yup	1.4.0	Esquemas de validación

UI y estilos

Tecnología	Versión	Propósito
Bootstrap	5.3.3	Framework CSS base
Reactstrap	9.2.3	Componentes Bootstrap para React
SASS	1.77.6	Preprocesador CSS
Lucide React	0.542.0	Iconos SVG (principal)
React Icons	5.5.0	Librería de iconos adicional
Feather Icons React	0.7.0	Iconos Feather
React Toastify	10.0.5	Notificaciones toast
React Select	5.8.1	Selects avanzados con búsqueda
Flatpickr	4.6.13	Date/time picker
Simplebar React	3.2.6	Scrollbar personalizado
FilePond	4.32.8	Upload de archivos con preview

Tablas y gráficas

Tecnología	Versión	Propósito
TanStack React Table	8.20.5	Tablas headless con ordenamiento/filtros
ApexCharts + React-ApexCharts	4.1.0 / 1.6.0	Gráficas interactivas
Nivo	0.99.0	Gráficas SVG (bar, line, pie, radar)
FullCalendar	6.1.19	Vista de calendario

Enrutamiento y navegación

Tecnología	Versión	Propósito
React Router DOM	6.26.2	Enrutamiento declarativo (v6)

Internacionalización

Tecnología	Versión	Propósito
i18next	23.15.2	Motor de traducciones
react-i18next	15.0.2	Integración con React (hook <code>useTranslation</code>)
i18next-browser-languagedetector	8.0.0	Detección automática del idioma del navegador

Otras utilidades

Tecnología	Versión	Propósito
Firebase	10.14.1	SDK incluido (configurado pero sin uso activo en producción)
Moment.js	2.30.1	Manejo de fechas
File Saver	2.0.5	Descarga de archivos en el navegador
React PDF Viewer	3.12.0	Visualización de PDFs inline
Cleave.js	1.6.0	Máscaras de input
React Markdown	10.1.0	Renderizado de Markdown (manual de usuario)
AOS	2.3.4	Animaciones al hacer scroll

3. Requisitos del Entorno

Para desarrollo

Requisito	Versión mínima recomendada
Node.js	18 LTS o 20 LTS
npm	Incluido con Node (no se usa yarn)
Git	Cualquier versión reciente

Requisito	Versión mínima recomendada
Editor	VS Code (configuración de debug incluida)

Nota: CRA 5 es compatible con Node 14+, pero se recomienda Node 18 LTS por LTS activo y compatibilidad con dependencias modernas.

Para producción (servidor)

Requisito	Descripción
Servidor web	Nginx, Apache, o CDN (S3 + CloudFront, Vercel, Netlify, etc.)
Rewrite de rutas	Obligatorio: todas las rutas deben redirigir a <code>index.html</code> (SPA)
HTTPS	Recomendado en producción
Backend API	Servidor REST accesible desde el frontend + soporte WebSocket (Socket.IO)

4. Configuración del Entorno de Desarrollo

1. Clonar el repositorio

```
git clone <url-del-repositorio>
cd MFarm-Client
```

2. Instalar dependencias

```
npm install
```

No ejecutar `npm run eject` bajo ninguna circunstancia — es irreversible y rompería la configuración del proyecto.

3. Configurar variables de entorno

Crear el archivo `.env` en la raíz del proyecto (ver sección [Variables de Entorno](#)):

```
cp .env.example .env # si existe, o crearlo manualmente
```

4. Iniciar el servidor de desarrollo

```
npm start
```

La aplicación se abre automáticamente en `http://localhost:3000`.

Scripts disponibles

Comando	Descripción
<code>npm start</code>	Servidor de desarrollo con hot-reload en <code>localhost:3000</code>
<code>npm run build</code>	Bundle de producción optimizado en la carpeta <code>/build</code>
<code>npm test</code>	Suite de tests con Jest + React Testing Library
<code>npm run eject</code>	NO USAR — expone la configuración interna de Webpack (irreversible)

Debug en VS Code

El proyecto incluye configuración de debug en `.vscode/launch.json`. Para depurar:

1. Iniciar el servidor de desarrollo (`npm start`)
2. En VS Code: `Run > Start Debugging` (o `F5`)
3. Se abre una instancia de Microsoft Edge con sourcemaps habilitados

5. Variables de Entorno

Las variables de entorno en CRA **deben empezar con el prefijo `REACT_APP_`** para ser accesibles en el código del cliente. El archivo `.env` está excluido del repositorio por `.gitignore`.

Variables disponibles

Variable	Requerida	Valor por defecto	Descripción
<code>REACT_APP_API_URL</code>	Sí	<code>http://localhost:3001</code>	URL base del backend REST y WebSocket
<code>REACT_APP_DEMO_MODE</code>	No	<code>false</code>	Si es <code>true</code> , reemplaza el login con un selector de roles para demostración

Archivos por entorno

Archivo	Cuándo se usa
<code>.env</code>	Variables locales de desarrollo (ignorado por git)
<code>.env.local</code>	Sobreescribe <code>.env</code> para la máquina local (ignorado por git)
<code>.env.production</code>	Variables para el build de producción
<code>.env.development</code>	Variables exclusivas del servidor de desarrollo

Importante: Las variables se inyectan en tiempo de build, no en tiempo de ejecución. Cambiar una variable requiere volver a hacer `npm run build`.

Ejemplo de `.env` para desarrollo

```

REACT_APP_API_URL=http://localhost:3001
REACT_APP_DEMO_MODE=false

```

Ejemplo de `.env.production`

```

REACT_APP_API_URL=https://api.midominio.com
REACT_APP_DEMO_MODE=false

```

Cómo se consumen las variables

Las variables se centralizan en `src/config.ts`:

```

// src/config.ts
const config = {
  api: {
    API_URL: process.env.REACT_APP_API_URL || "http://localhost:3001",
  },
};
export default config;

```

Este objeto `config.api.API_URL` es la única fuente de verdad para la URL del backend. No se debe leer `process.env.REACT_APP_API_URL` directamente desde los componentes.

6. Estructura del Proyecto

```

MFarm-Client/
├── public/
│   ├── index.html          # Template HTML principal (script de tema dark
mode)
│   ├── manifest.json       # PWA manifest
│   └── logo.png            # Favicon e ícono
├── src/
│   ├── App.tsx             # Raíz: ConfigContext + Route +
PeriodClosedModal
│   ├── index.tsx           # Bootstrap: Redux store + BrowserRouter + App
│   └── config.ts           # Configuración centralizada (API URL, Google,
Facebook)
├── i18n.ts                 # Configuración de i18next
└── Routes/
    ├── allRoutes.tsx       # Definición de todas las rutas de la app
    ├── AuthProtected.tsx   # Guard: redirige a /login si no hay sesión
    └── RoleProtected.tsx   # Guard: redirige a / si el rol no está

```

```

permitido
├── index.tsx          # Componente Routes (aplica los guards)
├── Layouts/
│   ├── Header.tsx
│   ├── Sidebar.tsx
│   ├── LayoutMenuData.tsx # Estructura del menú lateral
│   ├── NonAuthLayout.tsx  # Layout sin sidebar (login, errores)
│   └── index.tsx          # VerticalLayout (app autenticada)
├── pages/              # Páginas por dominio de negocio
│   ├── Authentication/
│   ├── Births/
│   ├── Configurations/
│   ├── Expenses/
│   ├── Farms/
│   ├── Feeding/
│   ├── Finance/
│   ├── Gestation/
│   ├── Groups/
│   ├── Home/
│   ├── Incomes/
│   ├── Inventory/
│   ├── Laboratory/
│   ├── Lactation/
│   ├── Medication/
│   ├── Orders/
│   ├── Outcomes/
│   ├── Pigs/
│   ├── Products/
│   ├── PurchaseOrders/
│   ├── Replacement/
│   ├── Reports/
│   ├── Sales/
│   ├── Subwarehouse/
│   ├── Suppliers/
│   ├── UserManual/
│   └── Users/
├── Components/Common/
│   ├── Details/          # Vistas de detalle reutilizables
│   ├── Filters/          # Componentes de filtros
│   └── Forms/             # Formularios Formik reutilizables (modales y
standalone)
│   ├── Graphics/         # Wrappers de charts (Nivo, ApexCharts)
│   ├── Lists/            # Componentes de lista
│   └── Shared/           # Tarjetas compartidas (FeedingPackagesCard,
etc.)
│   ├── Tables/           # CustomTable, Pagination, TanStack wrappers
│   └── Views/            # Layouts de vista
├── slices/              # Redux Toolkit: un slice por dominio
│   ├── index.ts         # combineReducers (rootReducer)
│   └── auth/            # Login, Register, ForgetPassword, Profile

```

```

├── ai/ # Chat IA
├── configurations/ # Configuración global y por granja
├── layouts/ # Preferencias de layout (tema, sidebar)
├── notifications/ # Notificaciones en tiempo real
├── periodClosing/ # Gestión de cierre de periodo

├── helpers/
│   ├── api_helper.ts # Axios config + clase APIClient
│   ├── feeding_urls.ts # URLs del módulo de alimentación
│   ├── configurations_urls.ts # URLs de configuraciones
│   ├── period_closing_urls.ts # URLs de cierre de periodo
│   └── reports_url_helper.ts # Constructor dinámico de URLs de
reportes
│   ├── impersonation_helper.ts # Lógica de impersonación de granja
│   ├── income_error_helper.ts # Mapeo de errores de ingresos
│   └── socketService.ts # Conexión Socket.IO para
notificaciones
│   ├── hooks/
│   │   └── useGlobalConfig.ts # Lee la configuración global desde
Redux
│   ├── useGroupsByStage.ts # Filtra grupos por etapa productiva
│   ├── usePigFilters.ts # Filtros de cerdos
│   └── useReportScope.ts # Determina si el reporte es global o
por granja
│   ├── common/
│   │   └── data_interfaces.ts # Interfaces TypeScript de todos los
modelos
│   ├── user_roles.ts # Constantes de roles
│   ├── enums/ # Enums de dominio
│   └── data/ # Datos estáticos de referencia
│   ├── utils/
│   │   └── chartTransformers.ts # Transforma datos API a formato de
gráficas
│   ├── closingFormatters.ts # Formatea datos de cierre de periodo
│   ├── colorUtils.ts # Utilidades de color
│   └── formUtils.ts # Helpers de formularios
(preventEnterSubmit, etc.)
│   ├── intlHelpers.ts # Formateo de números y moneda
│   ├── logger.ts # Logger condicional (dev vs prod)
│   └── periodClosedEvents.ts # Eventos del modal de periodo cerrado
│   ├── locales/
│   │   └── sp.json # Traducciones en español (idioma
principal)
│   ├── en.json # Traducciones en inglés
│   ├── pt.json # Traducciones en portugués
│   └── manual-sp.json # Traducciones del manual de usuario
(ES)
│   ├── manual-en.json # Traducciones del manual de usuario
(EN)
│   └── manual-pt.json # Traducciones del manual de usuario

```



```
(PT)
├── .env # Variables de entorno locales (NO en git)
├── .gitignore
├── package.json
├── tsconfig.json
└── CLAUDE.md # Instrucciones para el asistente de IA
```

7. Arquitectura de la Aplicación

Flujo de arranque

```
index.tsx
├── Redux Provider (store)
│   └── BrowserRouter (basename = PUBLIC_URL)
│       └── App.tsx
│           ├── ConfigContext.Provider ← contexto global (apiUrl,
│           │   user, impersonación)
│           ├── Routes/index.tsx ← enrutador principal
│           └── PeriodClosedModal ← modal global de periodo
│               └── cerrado
```

ConfigContext — Contexto global de aplicación

App.tsx provee un contexto React (**ConfigContext**) disponible en toda la app:

Campo	Tipo	Descripción
apiUrl	string	URL base del backend (de REACT_APP_API_URL)
axiosHelper	APIClient	Instancia única del cliente HTTP (memoizada)
userLogged	object	Usuario autenticado con impersonación aplicada
setUserLogged	function	Actualiza el usuario en contexto
impersonation	ImpersonationData null	Granja actualmente impersonada por el Superadmin
setImpersonation	function	Activa o desactiva la impersonación
superadminFarmId	string	ID de la granja activa cuando el rol es Superadmin
setSuperadminFarmId	function	Cambia la granja activa del Superadmin

Para consumirlo en cualquier componente:

```
import { useContext } from "react";
import { ConfigContext } from "App";

const { axiosHelper, userLogged } = useContext(ConfigContext);
```

Patrón de página

Cada módulo sigue el mismo patrón:

```
pages/Modulo/
├── ViewModulo.tsx      ← página de listado (tabla principal)
├── ModuloDetails.tsx  ← vista de detalle de un registro
└── ...

Components/Common/Forms/
└── ModuloForm.tsx     ← formulario Formik en modal (Create/Edit)

slices/modulo/
├── reducer.ts         ← Redux slice (state + actions)
└── thunk.ts           ← thunks async (llamadas a la API)
```

Comunicación en tiempo real

El módulo de notificaciones usa Socket.IO ([src/helpers/socketService.ts](#)):

- Se conecta al namespace `/notifications` del backend
- Autenticación vía `{ auth: { token } }` (el mismo token JWT de la sesión)
- Eventos manejados: `notification:new` (nueva notificación), `connect_error`
- Al recibir una notificación, despacha acciones Redux (`addNotification`, `fetchUnreadCount`)
- La conexión se inicia dentro de `AuthProtected.tsx` al detectar una sesión válida

8. Módulos y Rutas

Rutas públicas (sin autenticación)

Ruta	Componente	Descripción
<code>/login</code>	<code>Login</code>	Inicio de sesión
<code>/logout</code>	<code>Logout</code>	Cierra sesión y redirige
<code>/auth-404-alt</code>	<code>Alt404</code>	Página de error 404
<code>/auth-offline</code>	<code>Offlinepage</code>	Sin conexión
<code>*</code>	<code>NotFound</code>	Cualquier ruta no encontrada

Rutas protegidas — Módulos principales

Granja y usuarios

Ruta	Descripción
/home	Panel principal (dashboard)
/farms/view_farms	Listado de granjas
/farms/farm_details/:farm_id	Detalle de una granja
users/view_users	Gestión de usuarios

Almacén (Warehouse)

Ruta	Descripción
/warehouse/inventory/view_inventory	Inventario principal
/warehouse/inventory/product_details	Detalle de producto en inventario
/warehouse/suppliers/view_suppliers	Proveedores
/warehouse/incomes/view_incomes	Ingresos de mercancía
/warehouse/outcomes/view_outcomes	Salidas de mercancía
/warehouse/products/product_catalog	Catálogo de productos

Sub-almacén

Ruta	Descripción
/subwarehouse/view_subwarehouse	Listado de sub-almacenes
/subwarehouse/subwarehouse_details/:id_subwarehouse	Detalle de un sub-almacén
/subwarehouse/subwarehouse_inventory	Inventario del sub-almacén
/subwarehouse/subwarehouse_incomes	Ingresos al sub-almacén
/subwarehouse/subwarehouse_outcomes	Salidas del sub-almacén

Órdenes y compras

Ruta	Descripción
/orders/send_orders	Envío de órdenes al almacén central
/orders/order_details/:id_order	Detalle de una orden
/orders/complete_order/:id_order	Completar una orden
/purchase_orders/view_purchase_orders	Órdenes de compra
/expenses/view_expenses	Gastos

Finanzas

Ruta	Descripción
/finance/period-closing	Lista de cierres de periodo
/finance/period-closing/:closingId	Detalle de un cierre de periodo

Producción porcina

Ruta	Descripción
/pigs/view_pigs	Listado de cerdos activos
/pigs/pig_details/:pig_id	Detalle de un cerdo
/pigs/discarded_pigs	Cerdos descartados
/pigs/inventory_pigs	Inventario de cerdos
/groups/view_groups	Todos los grupos
/groups/view_weaned_groups	Grupos en etapa de destete
/groups/view_growing_groups	Grupos en crecimiento
/groups/view_finishing_groups	Grupos en finalización
/groups/view_exit_groups	Grupos dados de baja
/groups/group_details/:group_id	Detalle de un grupo

Reproducción

Ruta	Descripción
/gestation/view_inseminations	Inseminaciones
/gestation/insemination_details/:insemination_id	Detalle de inseminación
/gestation/view_pregnancies	Gestaciones activas
/births/view_births	Partos registrados
/births/view_upcoming_births	Partos próximos
/lactation/view_litters	Camadas en lactación
/lactation/litter_details/:litter_id	Detalle de camada
/replacement/view_sows	Hembras de reemplazo
/replacement/view_boars	Machos de reemplazo

Laboratorio

Ruta	Descripción
/laboratory/extractions/view_extractions	Extracciones de semen
/laboratory/samples/view_samples	Muestras
/laboratory/samples/sample_details/:sample_id	Detalle de muestra

Alimentación y medicación

Ruta	Descripción
/feeding/view_feeding_packages	Paquetes / recetas de alimentación
/feeding/view_feed_preparations	Preparaciones de alimento
/feeding/view_feeding_consumption	Consumo de alimento
/medication/view_medication_package	Paquetes de medicación
/medication/view_vaccination_plans	Planes de vacunación

Ventas

Ruta	Descripción
/sale/view_sale_groups	Grupos en proceso de venta
/sale/view_sold_groups	Grupos vendidos
/sale/view_pig_sales	Venta individual de cerdos

Reportes

Categoría	Rutas disponibles
Producción	inseminaciones-partos, grupos, mortalidad, peso-alimento, reproductivo
Inventario	movimientos, consumo de alimento, alertas, valorización
Finanzas	compras, análisis de costos, rentabilidad, cierre de operaciones, flujo de caja, estado de proveedor, gastos
Ventas	resumen, clientes
Catálogos	/reports/catalogs
Trazabilidad	/reports/traceability
Auditoría	/reports/audit

Configuraciones y utilidades

Ruta	Rol requerido	Descripción
/configurations/global	Superadmin	Configuración global del sistema
/configurations/farm	farm_manager	Configuración de la granja asignada
/user-manual	Cualquiera	Manual de usuario integrado

9. Gestión de Estado (Redux)

Arquitectura de slices

El estado global usa **Redux Toolkit**. Cada dominio tiene su propia carpeta en `src/slices/` con dos archivos:

```
slices/dominio/
├── reducer.ts    ← createSlice: define state, actions y reducers
síncronos
└── thunk.ts     ← funciones async que llaman a la API y despachan
actions
```

Los thunks siguen la convención de funciones curried (no `createAsyncThunk`):

```
// Patrón de thunk en este proyecto
export const fetchItems = (params: any) => async (dispatch: AppDispatch) =>
{
  try {
    const response = await api.get(URL, params);
    dispatch(setItems(response.data));
  } catch (error) {
    dispatch(setError(error));
  }
};
```

Slices registrados en el store

Clave en store	Módulo	Descripción
Layout	layouts/	Preferencias de tema, sidebar, layout
Login	auth/login/	Estado de autenticación (token, usuario)
Account	auth/register/	Registro de cuenta
ForgetPassword	auth/forgetpwd/	Recuperación de contraseña
Profile	auth/profile/	Datos del perfil del usuario
Notifications	notifications/	Notificaciones en tiempo real

Clave en store	Módulo	Descripción
ai	ai/	Estado del chat con IA
Configurations	configurations/	Configuración global y por granja
PeriodClosing	periodClosing/	Gestión de cierre de periodo financiero

Agregar un nuevo slice

1. Crear la carpeta `src/slices/nuevo_modulo/`
2. Crear `reducer.ts` con `createSlice`
3. Crear `thunk.ts` con las funciones async
4. Registrar el reducer en `src/slices/index.ts`

10. Autenticación y Autorización

Flujo de autenticación

1. El usuario ingresa credenciales en `/login`
2. El backend retorna un JWT token
3. El token se almacena en `sessionStorage` bajo la clave `"authUser"` como JSON: `{ token: "..."` }
4. `setAuthorization(token)` actualiza el header `Authorization: Bearer <token>` en Axios globalmente
5. Al recargar la página, `api_helper.ts` lee el token de `sessionStorage` al inicializarse

Importante: `sessionStorage` se borra al cerrar el tab del navegador. Esto es intencional por seguridad.

Guards de ruta

AuthProtected (`src/Routes/AuthProtected.tsx`):

- Verifica que exista `sessionStorage["authUser"]` con un token válido
- Si no hay token → redirige a `/login`
- Si hay token → conecta el socket de notificaciones y carga `globalConfig` si no está cargado
- Envuelve todas las rutas protegidas

RoleProtected (`src/Routes/RoleProtected.tsx`):

- Recibe un array de `allowedRoles`
- Verifica que el rol del usuario esté en esa lista
- Si no tiene el rol → redirige a `/` (home)
- Actualmente usado en `/configurations/global` (solo `Superadmin`) y `/configurations/farm` (solo `farm_manager`)

Roles de usuario

Los roles están definidos en `src/common/user_roles.ts`. Los roles principales son:

Rol	Descripción
Superadmin	Acceso total, puede impersonar cualquier granja
farm_manager	Administrador de una granja específica
(otros roles)	Ver <code>src/common/user_roles.ts</code> para el listado completo

Impersonación (Superadmin)

El Superadmin puede "ver la aplicación como" cualquier granja usando la funcionalidad de impersonación:

- Estado guardado en `sessionStorage["impersonation"]` y `sessionStorage["superadmin_farm_id"]`
- `getEffectiveUser()` en `impersonation_helper.ts` retorna el usuario con `farm_assigned` y `role` sobreescritos
- El contexto global (`ConfigContext`) expone `impersonation` y `setImpersonation`
- Los componentes deben usar `userLogged` del contexto (no leer `sessionStorage` directamente) para tener el usuario efectivo con impersonación aplicada

11. Cliente HTTP (Axios)

Configuración base

El cliente HTTP está centralizado en `src/helpers/api_helper.ts`. La clase `APIClient` encapsula todos los métodos:

```
import { APIClient } from "helpers/api_helper";
const api = new APIClient();

// Ejemplos de uso
const data = await api.get("/endpoint", { param: value });
const created = await api.create("/endpoint", payload);
const updated = await api.update("/endpoint", payload);
await api.delete("/endpoint");
```

Métodos disponibles

Método	HTTP	Descripción
<code>get(url, params?)</code>	GET	Petición con query params opcionales
<code>getBlob(url, config?)</code>	GET	Respuesta tipo Blob (descarga de archivos)
<code>create(url, data)</code>	POST	Crear un recurso (body JSON)
<code>postBlob(url, data, config?)</code>	POST	POST con respuesta Blob
<code>uploadImage(url, file)</code>	POST	Upload de archivo con <code>multipart/form-data</code>

Método	HTTP	Descripción
<code>update(url, data)</code>	PATCH	Actualización parcial de un recurso
<code>put(url, data)</code>	PUT	Reemplazo completo de un recurso
<code>delete(url, config?)</code>	DELETE	Eliminación de un recurso

Interceptor de periodo cerrado

Existe un interceptor de respuesta que captura el error HTTP 409 con `error.error === "PERIOD_CLOSED"`:

- Emite un evento global que abre el `PeriodClosedModal`
- Marca el error con `__periodClosed = true`
- Los componentes pueden usar `isPeriodClosedError(err)` para no mostrar UI de error duplicada

Organización de URLs

Las URLs no están en un archivo único, sino distribuidas por dominio:

Archivo	Contiene
<code>helpers/feeding_urls.ts</code>	Endpoints de alimentación (paquetes, preparaciones, administración)
<code>helpers/configurations_urls.ts</code>	Endpoints de configuración global y por granja
<code>helpers/period_closing_urls.ts</code>	Endpoints de cierre de periodo
<code>helpers/reports_url_helper.ts</code>	Constructor dinámico de URLs de reportes (JSON, PDF, Excel)

Las URLs del resto de módulos se definen directamente en sus respectivos `thunk.ts`.

Constructor de URLs de reportes

```
import { buildReportUrl } from "helpers/reports_url_helper";

const url = buildReportUrl({
  apiUrl: config.api.API_URL,
  basePath: "reports/production/groups",
  isGlobal: false,
  farmId: "123",
  variant: "pdf", // "json" | "pdf" | "excel"
  query: { from: "2026-01-01", to: "2026-06-30" },
});
```

Agrega automáticamente el parámetro `lang` basado en el idioma activo del usuario (`localStorage["I18N_LANGUAGE"]`).

12. Internacionalización (i18n)

Idiomas soportados

Código interno	Idioma	Archivo de traducciones
sp	Español (principal)	src/locales/sp.json
en	Inglés	src/locales/en.json
pt	Portugués (Brasil)	src/locales/pt.json

Uso en componentes

```
import { useTranslation } from "react-i18next";

const MyComponent = () => {
  const { t } = useTranslation();
  return <button>{t("common.save")}</button>;
};
```

Reglas de localización (obligatorias)

1. **Ningún texto en el JSX puede estar hardcodeado.** Todo texto visible al usuario pasa por `t()`.
2. **Agregar la clave en los tres archivos** (`sp.json`, `en.json`, `pt.json`) antes de usarla.
3. **Las traducciones deben sonar nativas**, no ser traducciones literales. Para EN: terminología de la industria porcina en inglés. Para PT: vocabulario de suinocultura brasileña.
4. Para interpolación: `t("key", { val: valor })` y `{{val}}` en el JSON.
5. Para HTML embebido: usar el componente `<Trans>`.

Cambio de idioma

El idioma se detecta automáticamente del navegador y se persiste en `localStorage["I18N_LANGUAGE"]`. El usuario puede cambiarlo desde la interfaz. El mapeo es: `es*` → `sp`, `en*` → `en`, `pt*` → `pt`.

13. Convenciones de Código

Nombrado

Elemento	Convención	Ejemplo
Componentes React	PascalCase	<code>BirthForm.tsx</code> , <code>GroupDetails.tsx</code>
Carpetas de componentes	PascalCase	<code>Components/Common/Forms/</code>

Elemento	Convención	Ejemplo
Hooks personalizados	camelCase con prefijo <code>use</code>	<code>useGroupsByStage.ts</code>
Slices Redux	camelCase	<code>reducer.ts</code> , <code>thunk.ts</code>
Interfaces TypeScript	PascalCase con <code>I</code> prefijo o sin él	<code>export interface Pig { ... }</code>
Identificadores en código	English	<code>pigId</code> , <code>farmName</code> , <code>birthDate</code>

Formularios

Los formularios usan el hook `useFormik` con esquema `Yup.object()`:

```
const formik = useFormik({
  initialValues: { name: "", weight: 0 },
  validationSchema: Yup.object({
    name: Yup.string().required(t("validation.required")),
    weight: Yup.number().positive().required(),
  }),
  onSubmit: (values) => { /* llamar thunk */ },
});
```

Modales de feedback

Después de operaciones CRUD, usar `SuccessModal` y `ErrorModal` (no `alert()`):

```
import SuccessModal from "Components/Common/Modals/SuccessModal";
import ErrorModal from "Components/Common/Modals/ErrorModal";
```

Imports absolutos

`tsconfig.json` tiene `baseUrl: "./src"`, por lo que los imports pueden ser absolutos:

```
// Correcto
import { APIClient } from "helpers/api_helper";
import { Pig } from "common/data_interfaces";

// Evitar (relativo desde profundidad)
import { APIClient } from "../../../helpers/api_helper";
```

TypeScript estricto

El proyecto usa `strict: true`. No se permiten:

- Variables `any` sin justificación

- Propiedades opcionales sin verificación de null
- Type assertions sin motivo (**as any**)

14. Flujo de Trabajo Git

Ramas

Rama	Uso
main	Código estable y desplegable. No se hace commit directo aquí.
dev	Rama de desarrollo activa. Todo el trabajo va aquí.
feature/nombre	Funcionalidades grandes se pueden ramificar desde dev

Versionado (SemVer — obligatorio)

El archivo **package.json** debe actualizarse en **el mismo commit** que los cambios:

Tipo de cambio	Versión a incrementar	Ejemplo
Corrección de bug, cambio menor	PATCH (x.x.+1)	1.26.1 → 1.26.2
Funcionalidad nueva, sin romper nada	MINOR (x.+1.0)	1.26.1 → 1.27.0
Cambio que rompe compatibilidad, rediseño	MAJOR (+1.0.0)	1.26.1 → 2.0.0

Merge a main

Cuando el código en **dev** está listo para producción:

```
git checkout main
git merge dev
git push origin main
git checkout dev # Volver a dev inmediatamente
```

15. Manual de Despliegue en Producción

Paso 1: Preparar variables de entorno

Crear el archivo **.env.production** en la raíz del proyecto con los valores del entorno productivo:

```
REACT_APP_API_URL=https://api.tudominio.com
REACT_APP_DEMO_MODE=false
```

Las variables se inyectan en tiempo de **build**, no de ejecución. Cada vez que cambies una variable debes volver a ejecutar el build.

Paso 2: Instalar dependencias

```
npm install
```

Paso 3: Generar el bundle de producción

```
npm run build
```

Esto genera la carpeta `/build` con todos los assets estáticos optimizados (HTML, JS, CSS minificados, con hashing de nombres para cache-busting).

Paso 4: Servir los archivos estáticos

El contenido de la carpeta `/build` debe servirse desde un servidor web estático. **La configuración más importante:** cualquier ruta que no sea un archivo estático debe retornar `index.html` (requerido por React Router con `BrowserRouter`).

Opción A: Nginx

```
server {  
    listen 80;  
    server_name tudominio.com;  
    root /var/www/mfarm-client/build;  
    index index.html;  
  
    # Redirigir todas las rutas a index.html (SPA)  
    location / {  
        try_files $uri $uri/ /index.html;  
    }  
  
    # Cache de assets estáticos (tienen hash en el nombre)  
    location /static/ {  
        expires 1y;  
        add_header Cache-Control "public, immutable";  
    }  
}
```

Opción B: Apache (.htaccess)

```
Options -MultiViews  
RewriteEngine On  
RewriteCond %{REQUEST_FILENAME} !-f  
RewriteRule ^ index.html [QSA,L]
```

Opción C: Vercel / Netlify

Plataformas como Vercel y Netlify detectan automáticamente proyectos CRA y configuran el rewrite de SPA. Solo es necesario:

1. Conectar el repositorio
2. Configurar las variables de entorno en el dashboard de la plataforma
3. El build command es `npm run build` y el output directory es `build`

Opción D: AWS S3 + CloudFront

1. Subir el contenido de `/build` al bucket S3
2. En CloudFront: configurar una Custom Error Response para código 403/404 → `/index.html` con status 200

Paso 5: Verificar CORS en el backend

El servidor del backend debe permitir peticiones desde el dominio del frontend. Verificar que el header `Access-Control-Allow-Origin` incluya el origen del frontend.

Paso 6: Verificar WebSocket

Socket.IO requiere que el servidor proxy (Nginx, etc.) soporte WebSockets:

```
# En el bloque location del backend/API proxy
location /socket.io/ {
    proxy_pass http://backend:3001;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "upgrade";
    proxy_set_header Host $host;
}
```

Checklist de despliegue

Antes de publicar una nueva versión, verificar:

- ☐ `package.json` tiene la versión correcta (SemVer)
- ☐ `.env.production` tiene `REACT_APP_API_URL` apuntando al backend correcto
- ☐ `REACT_APP_DEMO_MODE=false`
- ☐ `npm run build` termina sin errores
- ☐ El servidor tiene configurado el rewrite de SPA (`try_files ... /index.html`)
- ☐ El backend tiene CORS configurado para el dominio del frontend
- ☐ El proxy soporta WebSocket para Socket.IO
- ☐ Verificar en producción: login, una operación CRUD básica, que el idioma se detecta correctamente

Consideraciones adicionales

Firebase: El SDK de Firebase está incluido en el bundle (`firebase ^10.14.1`). Las credenciales de Google y Facebook en `src/config.ts` están vacías — si no se usa autenticación social, esto es correcto y no afecta el funcionamiento.

Modo oscuro: La preferencia se persiste en `localStorage["layoutModeType"]`. Se aplica via un script inline en `index.html` antes del primer render de React, evitando el parpadeo de contenido (FOUC).

PWA: El `manifest.json` actual es el template por defecto de CRA. Si se desea publicar como PWA (instalable en móvil), actualizar `name`, `short_name`, íconos y colores en `public/manifest.json`.

Node version en CI/CD: Usar siempre Node 18 LTS o Node 20 LTS. CRA 5 puede dar warnings con Node 22+.

Documento generado para MFarm Client v1.26.1 — Junio 2026